Graphs

What is a Graph?

A graph is a mathematical and algorithmic model for representing relationships between objects. We write a graph as $G = (V, E)$, where V is the set of vertices (or nodes) and E is the set of edges connecting pairs of vertices.

Vertices represent entities, and edges represent relationships between those entities.

Graphs are useful whenever the structure of connections matters more than the objects in isolation. Typical examples include cities connected by roads, users connected in social networks, web pages connected by hyperlinks, and tasks connected by prerequisite constraints.

Main Types of Graphs

Directed vs Undirected

A graph is **directed** if each edge has a direction. An edge from u to v is written (u, v) and does not imply (v, u).

Directed graphs (digraphs) are useful for asymmetric relationships such as one-way streets, web links, prerequisite chains, and data-flow dependencies.

A graph is **undirected** if edges have no direction. If an edge connects u and v, the connection can be traversed both ways.

Undirected graphs are useful for mutual relationships such as friendship networks, two-way roads, or bidirectional communication links.

Weighted vs Unweighted

A graph is **weighted** if each edge e has a numerical weight $w(e)$ (for example distance, cost, time, capacity, or risk).

An **unweighted** graph has no explicit edge weights, so all edges are treated equally. You can view it as a weighted graph where every edge has the same weight (often 1).

Type Comparison

Type pair	Key distinction	Typical use
Directed vs undirected	Whether edges have orientation	One-way dependencies vs mutual relationships
Weighted vs unweighted	Whether edges carry numeric values	Optimization by cost vs pure connectivity

Common Terminology

Term	Meaning
Adjacent vertices	Two vertices with an edge between them
Degree (undirected)	Number of incident edges
In-degree (directed)	Number of incoming edges
Out-degree (directed)	Number of outgoing edges
Path	Sequence of vertices where consecutive vertices are connected
Cycle	Path that starts and ends at the same vertex
Connected graph (undirected)	Every pair of vertices has a path between them

Graph Representations

Two standard representations are adjacency lists and adjacency matrices.

Adjacency List

For each vertex, store a list of neighboring vertices.

For weighted graphs, each neighbor entry also stores a weight, for example (v, w).

Space complexity:

$$O(|V| + |E|)$$

This is usually preferred for sparse graphs and traversal-heavy algorithms.

Adjacency Matrix

Store edge information in a $|V| \times |V|$ matrix.

For unweighted graphs, matrix entries indicate edge existence.

For weighted graphs, matrix entries store edge weights (and a sentinel such as 0 or ∞ for no edge).

Space complexity:

$$O(|V|^2)$$

Edge lookup is $O(1)$, so this can be useful for dense graphs.

Representation Comparison

Representation	Space	Edge lookup	Best when
Adjacency list	$O(\|V\| + \|E\|)$	Typically $O(\deg(v))$ to scan neighbors	Graph is sparse; traversals are common
Adjacency matrix	$O(\|V\|^2)$	$O(1)$	Graph is dense; constant-time edge checks matter

Weighted Graphs (Detail)

In a weighted graph, edge weights allow optimization problems such as shortest path or minimum-cost connectivity.

![[weighted-graph.png]]

Example Weighted Adjacency Matrix

	0	1	2	3	4	5	6	7
0	0	6	8	2	0	0	0	0
1	6	0	3	0	4	0	0	0
2	8	3	0	4	2	6	0	0
3	2	0	4	0	0	0	7	0
4	0	4	2	0	0	1	0	1
5	0	0	6	0	1	0	2	7
6	0	0	0	7	0	2	0	5
7	0	0	0	0	1	7	5	0

![[weighted-graph-1.png]] ![[adjecency-list.png]]

Minimum Spanning Tree (MST)

For a connected weighted graph, a **spanning tree** is a connected, acyclic subgraph that includes all vertices.

A **minimum spanning tree** is a spanning tree with minimum possible total edge weight.

Common assumption for proofs: if all edge weights are distinct, the MST is unique.

![[minimum-spanning-tree.png]]

Applications

MSTs are used in network design problems where all points must be connected at minimum cost: computer networks, road systems, utility grids, circuits, cable infrastructure, and related optimization tasks.

Core Properties

Cut Property For any cut of the graph, the lightest edge crossing that cut is safe to include in an MST.

Proof intuition: if a heavier crossing edge were in an MST instead, swapping it with the lighter crossing edge would lower total weight, contradicting minimality.

![[cut-property.png]]

Cycle Property For any cycle, the heaviest edge on that cycle is not part of an MST (with distinct weights).

Proof intuition: removing the heaviest edge from a cycle and keeping a lighter alternative preserves connectivity and reduces total weight.

![[cycle-property.png]]

Prim's Algorithm

Prim's algorithm grows one tree from a start vertex by repeatedly adding the lightest edge from the current tree to a new vertex.

It is greedy, and correctness follows from repeatedly applying the cut property.

![[prims-algorithm.png]]

Prim: Idea

- Maintain a set of vertices already in the tree.
- For each outside vertex, track the lightest edge connecting it to the tree.
- Repeatedly add the vertex with minimum key value.

Prim: Pseudocode

```
PRIM(G, s)
    for each vertex v in V:
        key[v] = infinity
        parent[v] = null
    key[s] = 0

    P = priority queue containing all vertices in V, keyed by key[.]

    while P is not empty:
```

```
    u = EXTRACT-MIN(P)
    for each edge (u, v) in Adj[u]:
        if v is in P and w(u, v) < key[v]:
            parent[v] = u
            key[v] = w(u, v)
            DECREASE-KEY(P, v, key[v])
```

Prim: Running Time

Using $n = |V|$ and $m = |E|$:

Priority queue	INSERT	EXTRACT-MIN	DECREASE-KEY	Total
Array	$O(1)$	$O(n)$	$O(1)$	$O(n^2)$
Binary [[Heaps|heap]]	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(m \log n)$
Fibonacci [[Heaps|heap]]	$O(1)$ amortized	$O(\log n)$ amortized	$O(1)$ amortized	$O(m + n \log n)$

Kruskal's Algorithm

Kruskal's algorithm sorts edges by weight and repeatedly adds the lightest edge that does not create a cycle.

It maintains connected components with Union-Find (Disjoint Set Union).

![[kruskals-algorithm.png]]

Kruskal: Idea

- Start with each vertex in its own component.
- Process edges from lightest to heaviest.
- Add an edge only if it connects two different components.

Kruskal: Pseudocode

```
KRUSKAL(G)
    A = empty set of edges

    for each vertex v in V:
        MAKE-SET(v)

    sort edges E by nondecreasing weight

    for each edge (u, v) in sorted E:
        if FIND(u) != FIND(v):
```

```
            A = A union {(u, v)}
            UNION(u, v)
            if |A| = |V| - 1:
                break

    return A
```

Kruskal: Running Time

Dominated by sorting plus near-linear Union-Find operations:

$$O(|E| \log |E|) = O(|E| \log |V|)$$

Prim vs Kruskal (At a Glance)

Algorithm	Grows	Typical data structure focus	Often preferred when
Prim	One tree from a start vertex	Priority queue over vertices	Graph is represented by adjacency lists and you want tree-growth behavior
Kruskal	Forest merged by edges	Sorting + Union-Find	Edge list is natural and sorting edges is straightforward

Why Graphs Matter in Algorithms

Many core algorithmic problems are graph problems: traversal, connectivity, shortest paths, dependency scheduling, and network optimization.

Related

- [[Searching Algorithms]]
- [[Amortized Analysis]]